
5G Network Slice Simulation

Three-part system architecture for dynamic 5G slicing
simulation

Akhil

IMT2022112

Anudeep

IMT2022013

Contents

- What is Intent-Based Networking (IBN)
- What is Network Slicing Technology
- Uses and Applications Overview
- Our Implementation Plan Strategy
- Architecture Overview
- AI & NLP Engine
- Database Integration
- Multi-Platform Support
- System Overview
- Influx DB → YAML Builder
- Simulation Engine
- Statistics Collector

— Intent-Based Networking

AI-Powered Network Management

Intent-Based Networking (IBN) is a software-enabled network administration architecture that uses artificial intelligence (AI), machine learning (ML), and network orchestration to configure networks based on business intent.

IBN transforms traditional hardware-centric, manual configured networks into programmable, software-based solutions that eliminate manual effort.

It allows network administrators to capture high-level business objectives, translate them into policies, and optimally configure the network through automated processes.

IBN Components

Translation & Validation

- Converts business intent into network actions
- Validates feasibility constraints
- Generates configuration designs
- Ensures accuracy

Automated Implementation

- Deploys policies automatically
- Configures network infrastructure
- Allocates required resources
- Enforces business rules

Assurance & Monitoring

- Continuous network monitoring
- Real-time validation
- Automated corrective actions
- Performance reporting

Network Slicing Uses

eMBB

Enhanced Mobile Broadband

- Video streaming with ultra-high definition
- Virtual and augmented reality applications
- Gaming with high bandwidth requirements

URLLC

Ultra-Reliable Low-Latency

- Autonomous vehicle communications
- Remote surgery and healthcare systems
- Industrial automation and robotics

mMTC

Massive Machine Type Comm

- Smart city sensor networks
- Environmental monitoring systems
- Connected home devices (IoT)

Uses & Applications

IBN APPLICATIONS

- Automated network policy management
- Real-time security threat response
- Dynamic resource allocation optimization

NETWORK SLICING APPLICATIONS

Critical Communications

Emergency services, hospitals, and public safety teams utilize ultra-reliable, low-latency slices for real-time data exchange in life-critical situations.

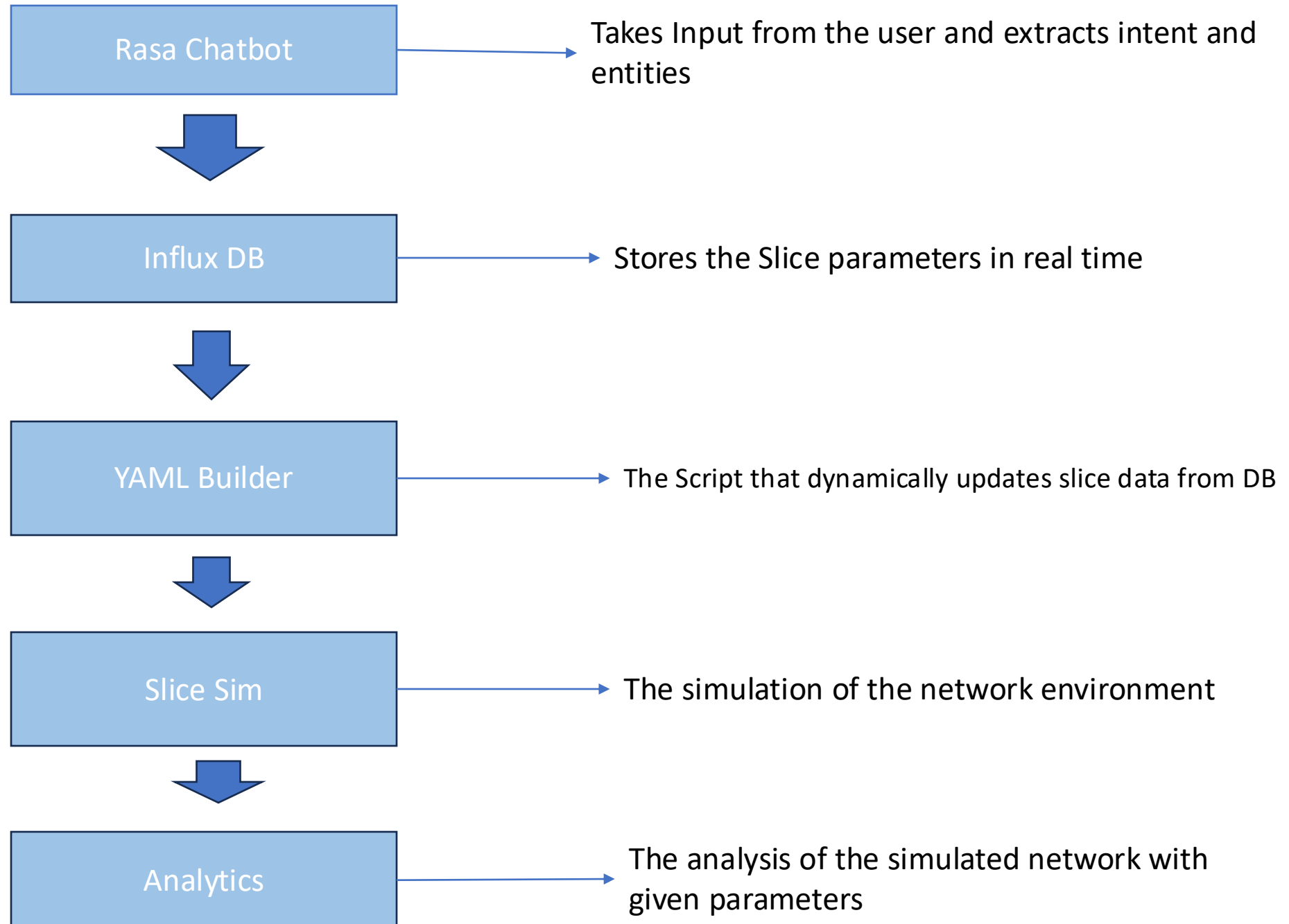
Vehicular & Drone Communications

Slices tailored for safe navigation, live telemetry, and coordination through low-latency, high-reliability links.

Optimized Streaming

Dedicated low-latency slices for OTT platforms (Netflix, YouTube) ensure buffer-free HD/4K delivery even during network congestion.

The Implementation Plan



Rasa Chatbot Overview (Chatbot)

Pipeline Foundation

Robust tokenization, regex patterns, lexical features, and count vector processing form the backbone of natural language understanding for network slice operations.

DIETClassifier Engine

100-epoch transformer neural network for intent classification and entity extraction with similarity constraints.

TEDPolicy Management

5-turn context-aware dialogue policy prediction for multi-step workflow orchestration.

AI Engine Capabilities

Intent Detection Strengths

- Handles greetings, slice requests, modifications, deletions, and feedback with 95% accuracy.

Entity Extraction

- Extracts bandwidth, latency, reliability, duration, service type, and slice parameters.

Technical Challenges

- Addresses network complexity, parameter validation, and real-time backend synchronization requirements.

Dialogue Opportunities

- Enables multi-step workflows, real-time corrections, and contextual clarifications for complex operations.

InfluxDB Integration

- **InfluxDB Backend Integration**

Real-time database storage for all network slice parameters, configurations, and operational metrics. The system maintains complete audit trails and enables historical analysis of slice performance and usage patterns.

- **Data Persistence**

All slice parameters, creation timestamps, and modification history are stored in a time-series format. This ensures data integrity and allows for precise reconstruction of network states.

Real-time Monitoring

Live tracking of slice performance metrics and resource utilization patterns provides immediate visibility into network health, facilitating automated scaling and optimization.

```

Your input -> Create a slice for video with 300 Mbps, 5 ms latency, 99.999% reliability_for_3_hours
Here's the slice summary:
- Bandwidth: 300 Mbps
- Latency: 5 ms
- Reliability: 99.999%
- Duration: 3 hours
- Service Type: video
Thanks! I'm processing your slice request now.
✅ Slice created successfully!
🆔 Slice ID: slice_0267af04
📺 Service: video
• Bandwidth: 300.0 Mbps
• Latency: 5.0 ms
• Reliability: 99.999%
• Duration: 3.0 hrs
📶 Remaining Available Bandwidth: 90.00 Mbps
💡 Use this Slice ID for future modifications or deletions.
Your input -> |

```

We give our input to chatbot which extracts the intents and saves it to the database.

```

name: network_slice
time      bandwidth dummy_tag duration latency reliability service_type slice_id      value
-----
1753374271736900800      init
1763098248800379000 250      3      5      99.999      gaming      slice_6934aa79
1763102983454611000 360      3      5      99.999      gaming      slice_9b4cd211
1763806774941107000 300      3      5      99.999      video      slice_0267af04
> |

```

We can see that the slice has been created with id slice_0267af04.

```
Command Prompt - rasa shel x + v
Implementing implicit namespace packages (as specified in PEP 420) is preferred to 'pkg_resources.declare_namespace'. See https://setuptools.pypa.io/en/latest/references/keywords.html#keyword-namespace-packages
declare_namespace(pkg)
C:\Users\mrxa\venv\lib\site-packages\tensorflow\lite\python\util.py:52: DeprecationWarning: jax.xla_computation is deprecated. Please use the AOT APIs.
from jax import xla_computation as _xla_computation
C:\Users\mrxa\venv\lib\site-packages\sanic_cors\extension.py:39: DeprecationWarning: distutils Version classes are deprecated. Use packaging.version instead.
SANIC_VERSION = LooseVersion(sanic_version)
2025-11-22 15:43:51 INFO root - Connecting to channel 'cmdline' which was specified by the '--connector' argument. Any other channels will be ignored. To connect to all given channels, omit the '--connector' argument.
2025-11-22 15:43:51 INFO root - Starting Rasa server on http://0.0.0.0:5005
2025-11-22 15:43:51 INFO rasa.core.processor - Loading model models\20251113-151640-irregular-experience.tar.gz...
2025-11-22 15:44:24 WARNING rasa.shared.utils.common - The Unexpected Intent Policy is currently experimental and might change or be removed in the future. Please share your feedback on it in the forum (https://forum.rasa.com) to help us make this feature ready for production.
2025-11-22 15:44:31 INFO root - Rasa server is up and running.
Bot loaded. Type a message and press enter (use '/stop' to exit):
Your input -> hi
Hello! I'm your Network Slicing Assistant. How can I help you today?
Your input -> delete slice slice_3acd702a
✅ Slice 'slice_3acd702a' deleted successfully.
Your input ->
```

We delete the slice by giving prompt to chatbot.

```
network_slice
> select * from network_slice
name: network_slice
time          bandwidth dummy_tag duration latency reliability service_type slice_id value
----          -
1753374271736900800 init          3          5          99.999 video slice_3acd702a 1
1763098220739180000 150          3          5          99.999 gaming slice_6934aa79
1763098248800379000 250          3          5          99.999 gaming slice_9b4cd211
1763102983454611000 360          3          5          99.999 gaming slice_9b4cd211
> select * from network_slice
name: network_slice
time          bandwidth dummy_tag duration latency reliability service_type slice_id value
----          -
1753374271736900800 init          3          5          99.999 gaming slice_6934aa79 1
1763098248800379000 250          3          5          99.999 gaming slice_9b4cd211
1763102983454611000 360          3          5          99.999 gaming slice_9b4cd211
> |
```

We can see that slice with id Slice_3acd702a has been deleted.

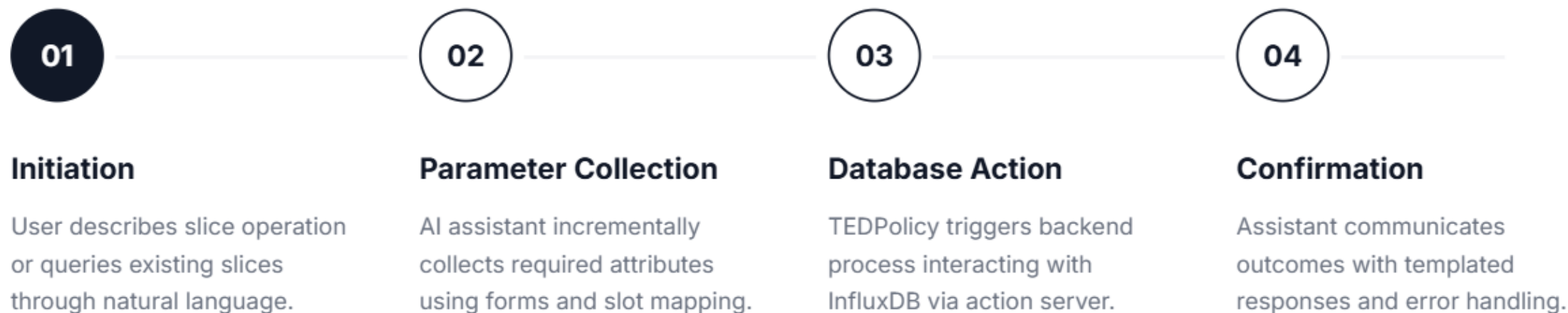
```
Your input -> modify slice slice_0267af04 bandwidth 300 Mbps to 200 Mbps
✓ Updated slice 'slice_9b4cd211': bandwidth changed from '360.0' to '200 Mbps'
Your input -> |
```

We modify the slice by giving prompt to chatbot.

```
name: network_slice
time          bandwidth dummy_tag duration latency reliability service_type slice_id      value
-----
1753374271736900800      init
1763098248800379000 250          3          5      99.999      gaming    slice_6934aa79
1763806774941107000 300          3          5      99.999      video     slice_0267af04
1763807192588117000 200          3          5      99.999      gaming    slice_9b4cd211
> |
```

We can see that the slice with id slice_94bcd211 has been modified with a change in bandwidth from 360 to 200 Mbps.

Operational Workflow Process



SliceSim Architecture

(Simulation)



Three-Part Integration

The system combines data ingestion, simulation execution, and analytics into a cohesive 5G network slicing platform.



Data Pipeline

Influx DB → YAML conversion pipeline ensures dynamic configuration and real-time adaptability.



Analytics Engine

Comprehensive metrics collection and reporting capability providing deep insights into simulation performance.

InfluxDB → YAML Builder

🎯 Purpose & Function

Converts slice information from Influx DB into YAML configuration files used by the simulator.

⚙️ Key Operations

- Connects to database
- Fetches slice entries
- Updates YAML with dynamic properties


```
C:\Users\mrxa>cd 5G-Network-Slicing

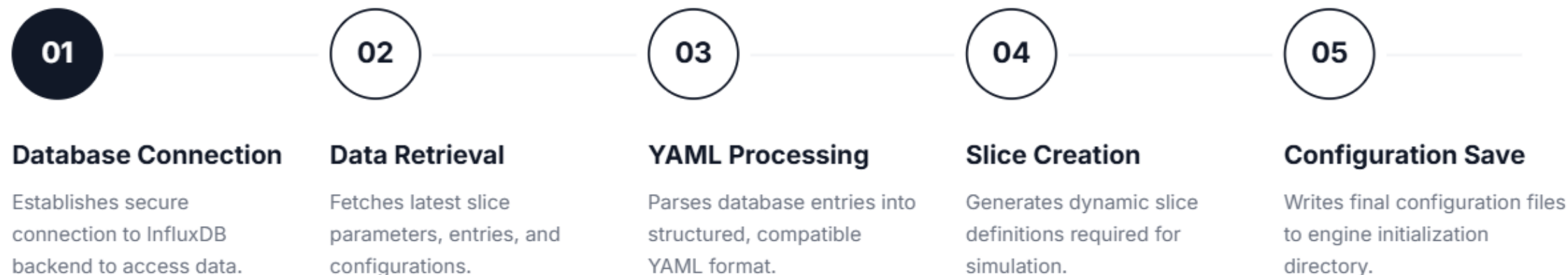
C:\Users\mrxa\5G-Network-Slicing>cd slicesim

C:\Users\mrxa\5G-Network-Slicing\slicesim>python sync_slices_from_db.py
✓Cleared old slices
✓Added slice: video_slice_unknown
✓Added slice: gaming_slice_slice_6934aa79
✓Added slice: video_slice_slice_0267af04
✓Added slice: gaming_slice_slice_9b4cd211
✓Updated base station ratios (4 slices)
✓Sync complete!

C:\Users\mrxa\5G-Network-Slicing\slicesim>
```

We sync the database with the yaml builder by running a python script which integrates the data so, the simulation can be run .

YAML Builder Workflow



Simulation Engine (main.py)

- **Core Function**

Transforms YAML configurations into Python objects and runs discrete-event simulation.

- **Framework**

Built on **SimPy** for accurate modelling of 5G network behaviour and resource allocation.

Simulation Object Creation

Base Stations

- Geographic position coordinates
- Signal coverage radius
- Total available bandwidth
- Associated slice objects

Network Slices

- Guaranteed bandwidth limits
- Maximum bandwidth capacity
- Quality of Service (QoS) settings
- Traffic pattern generators

Mobile Clients

- Initial position coordinates
- Mobility movement patterns
- Usage frequency models
- Active slice subscriptions

Simulation Loop Process



Client Movement

Clients move according to mathematical distributions (random, uniform, gaussian)



Coverage Detection

KD-Tree efficiently finds nearest base station and checks coverage boundaries



Connection Management

Handle connections, disconnections, and handovers based on client movement



Resource Allocation

Slices allocate bandwidth based on availability and client demand patterns

Statistics Collector

Purpose

- Collects comprehensive performance metrics in parallel with simulation execution, running as a background process to ensure data accuracy without interrupting the core engine.

Real-time Analysis

- Tracks connection success rates
- Monitors bandwidth usage patterns
- Analyzes client load distribution
- Evaluates overall network performance

Key Performance Metrics



Connection Ratios

SUCCESS RATE

Success ratio of client connections established

BLOCKING

Block ratio due to insufficient network resources

COVERAGE

Coverage ratio within base station range



Network Dynamics

MOBILITY

Handover ratio during client movement

LOAD BALANCE

Average slice load distribution across network

DISTRIBUTION

Per-slice client distribution analytics



Resource Utilization

BANDWIDTH

Total bandwidth consumption metrics

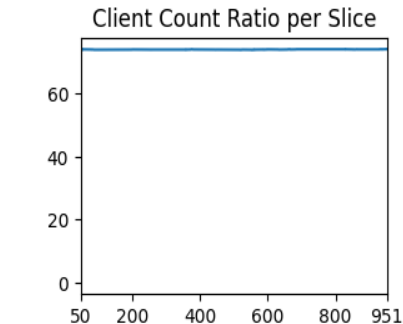
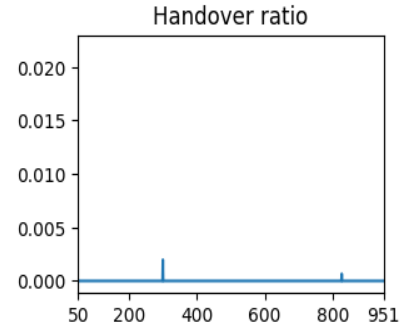
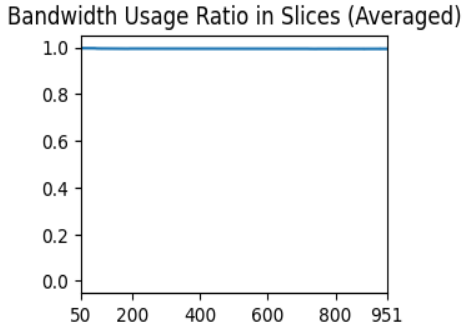
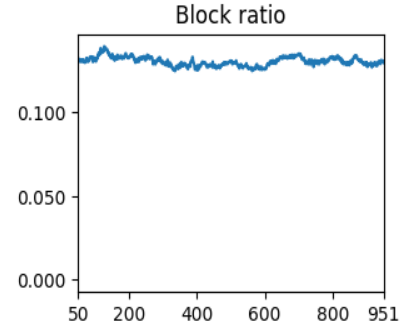
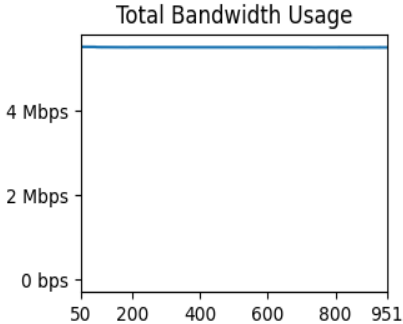
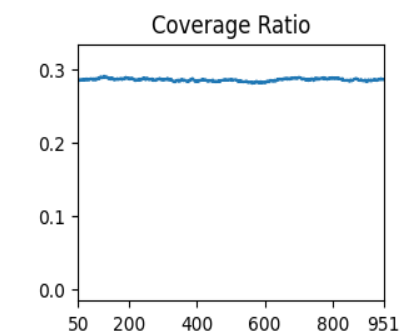
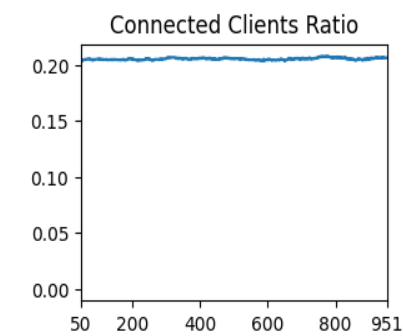
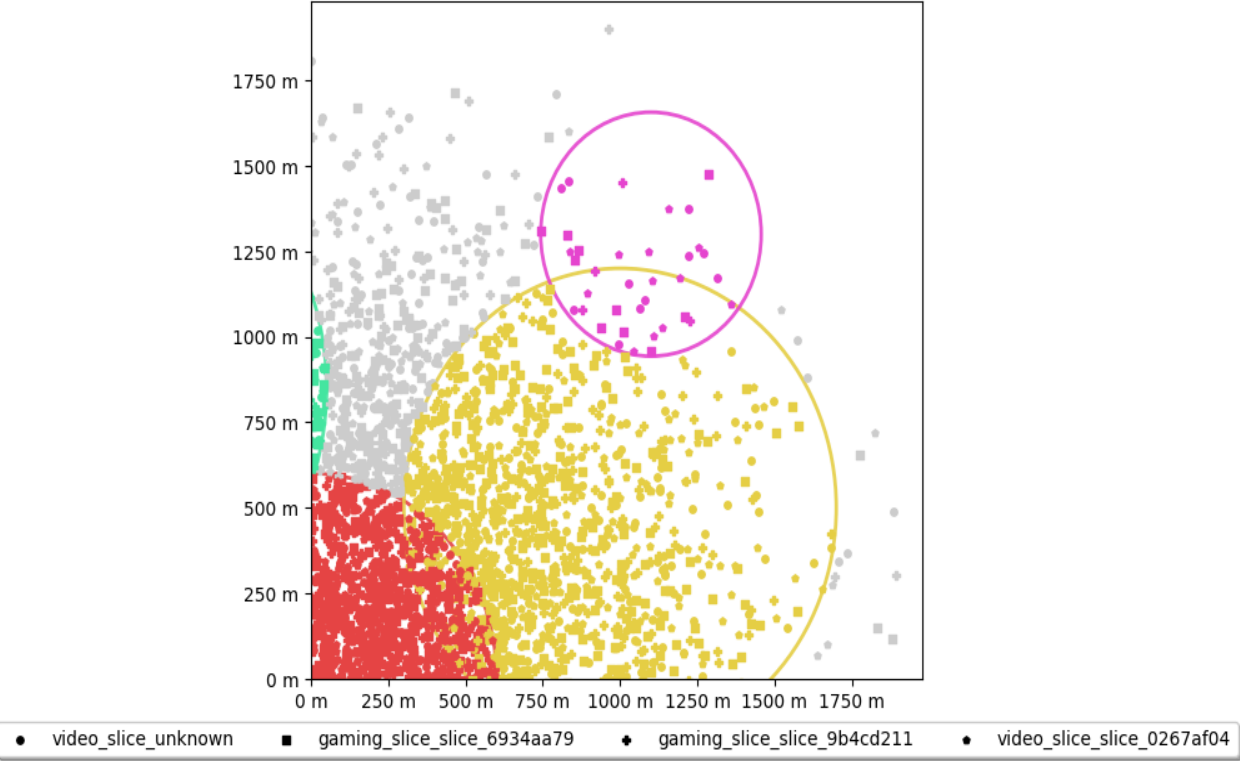
CAPACITY

Slice capacity utilization percentage

SUMMARY

End-of-simulation summary statistics

After running the simulator, we get the output graph as shown.



Initial number of clients	10000
Average connected clients	0.21
Average bandwidth usage	5.490 Mbps
Average load factor of slices	0.99
Average coverage ratio	0.29
Average block ratio	0.1303
Average handover ratio	0.0000

Slice Name	Total Clients	Connected Clients	Base stations Capacity (Mbps)	Used (Mbps)
video_slice_unknown	2500	2209	250000	2500
gaming_slice_6934aa79	2500	2199	250000	2499.83
video_slice_0267af04	2500	2223	250000	2083.33
gaming_slice_9b4cd211	2500	2215	250000	2500

We ran the simulation for 10,000 clients with the duration of the simulation $\approx$ 16.5 minutes.

System Integration Flow

01

Data Source

Influx DB stores dynamic slice configurations and network parameters.

02

Configuration Engine

YAML Builder creates simulation-ready configurations from database entries.

03

Simulation Runtime

SimPy engine processes client movement, connections, and resource allocation.

Technical Benefits

The three-part architecture provides dynamic configuration, accurate simulation, and comprehensive analytics for 5G network slicing.

Dynamic Configuration

- No manual YAML editing required
- Data-driven slice definitions
- Real-time configuration updates

Accurate Simulation

- Discrete-event modelling with SimPy
- Realistic client mobility patterns
- Precise resource allocation algorithms

Analytics & Insights

- Real-time metrics collection for network optimization
- Detailed reporting for network planning decisions
- Comprehensive performance summary

Thank You